# PCGT: A Graph Transformer with Topology-Aware Partition Attention

**Ranjan Kumar Choubey*** **Kushal Bose** **Swagatam Das**

Electronics and Communication Sciences Unit, Indian Statistical Institute, Kolkata, India

## Abstract

Graph Transformers for node classification must choose between global attention at $O(N^2)$ cost and efficient kernel approximations that discard topology. We propose PCGT (Partition-Conditioned Graph Transformer), which sidesteps this choice by operating at two attention resolutions. Exact softmax attention within graph partitions captures intra-community patterns, while learned seed vectors pool each partition into representatives for cross-partition global context. The model blends this partition-conditioned attention with a GCN backbone through learnable parameters—including an unconstrained self-connection weight $\beta$ that can go negative, suppressing a node's own features when the attention context is more informative.

On 14 benchmarks—including three large-scale graphs up to 2.4M nodes—PCGT outperforms SGFormer by +4.1% on Chameleon and +3.7% on Squirrel, while also improving on homophilic datasets (CiteSeer +0.5%, PubMed +0.7%) and scaling competitively to million-node graphs. PCGT ranks first or tied-first on 11 of 14 datasets. Ablations show that the attention mechanism's contribution grows with heterophily, and that the design choices—local attention, global seeds, partition encoding—each contribute on the most heterophilic benchmark, with mixed or negligible effects on homophilic graphs.

## 1 Introduction

Graph neural networks (GNNs) based on message passing [Gilmer et al., 2017, Kipf and Welling, 2017, Scarselli et al., 2009] have achieved strong results on node classification tasks, particularly on *homophilic* graphs where connected nodes share similar labels. However, many real-world graphs are *heterophilic*—neighboring nodes frequently belong to different classes [Zhu et al., 2020, Pei et al., 2020]. Standard GNNs struggle in this setting because neighborhood aggregation blends features from dissimilar nodes, degrading representations [Li et al., 2018].

Two families of approaches address this challenge. *Heterophily-specific GNNs* ($H_2$GCN [Zhu et al., 2020], GloGNN [Li et al., 2022], GPR-GNN [Chien et al., 2021]) modify the aggregation scheme to separate or reweight heterophilic neighbors, but remain constrained to local or fixed-hop neighborhoods. *Graph Transformers* (NodeFormer [Wu et al., 2022], SGFormer [Wu et al., 2023b], GraphGPS [Rampášek et al., 2022]) compute global all-pair attention to capture long-range dependencies, but either require $O(N^2)$ cost or use lossy kernel approximations that treat all node pairs uniformly regardless of graph topology.

Real-world graphs have multi-resolution structure: nodes form local communities (captured by graph partitions) while also participating in global patterns (cross-community interactions). Local GNNs ignore the global patterns; single-resolution global transformers ignore the community structure. A method that operates at both scales should do better.

We propose **PCGT** (Partition-Conditioned Graph Transformer), which operates at two attention resolutions simultaneously:

- **Fine (local):** Exact softmax attention within each graph partition, capturing detailed intra-community patterns at cost $O(N^2/K)$.
- **Coarse (global):** Learned seed vectors attention-pool each partition into $M$ representative vectors; every node then cross-attends to all $KM$ representatives for global context at cost $O(NMK)$.
- **Adaptive blending:** Learnable scalars $\alpha$ (local vs. global) and $\beta$ (unconstrained self-connection) allow the model to adapt to both homophilic and heterophilic regimes.

Our contributions are:

1. A **multi-resolution partition attention** mechanism that is both topology-aware and subquadratic,

*Corresponding author. Email: `ranjankumarchoubey999@gmail.com`. Code: `https://github.com/ranjanchoubey/PCGT`

combining the expressiveness of exact attention with the scalability of learned pooling.

2. An **unconstrained self-connection weight** $\beta$ that adapts per dataset—going negative on Film and Deezer where suppressing self-features helps, and strongly positive on feature-rich heterophilic graphs like Chameleon ($\beta = +3.08$) and Squirrel ($\beta = +3.53$).

3. **Experiments on 14 benchmarks**—including three large-scale graphs up to 2.4M nodes—where PCGT beats SGFormer by +4.1% on Chameleon and +3.7% on Squirrel, improves on CiteSeer (+0.5%) and PubMed (+0.7%), and ranks first or tied-first on 11 of 14 datasets.

4. An **analysis of partition strategies**: topology-aware (METIS) partitioning helps on structured heterophilic graphs, and ablations confirm that each component—local attention, global seeds, PSE, GCN branch—adds measurable accuracy.

Figure 1 illustrates the core idea. On a heterophilic graph with three communities, flat global attention (panel b) dilutes each node's signal across all $N$ nodes. PCGT (panel c) instead computes exact attention within each partition and routes cross-partition information through learned seed representatives, preserving community structure while maintaining global receptive field.

## 2 Related Work

**Message-Passing GNNs.** The graph neural network framework [Scarselli et al., 2009, Gilmer et al., 2017] aggregates features from local neighborhoods. GCN [Kipf and Welling, 2017] and GAT [Veličković et al., 2018] aggregate features from immediate neighbors, performing well on homophilic graphs. APPNP [Klicpera et al., 2019] decouples feature transformation from propagation using personalized PageRank. SIGN [Rossi et al., 2020] pre-computes multi-hop features for scalable inception-style aggregation. GIN [Xu et al., 2019] provides a maximally-expressive message passing framework. These methods are inherently local and struggle when informative nodes are distant or dissimilar to neighbors [Li et al., 2018].

**Heterophily-Specific GNNs.** $H_2$GCN [Zhu et al., 2020] addresses heterophily through ego/neighbor separation, higher-order neighborhoods, and intermediate representation combination. GloGNN [Li et al., 2022] learns a global coefficient matrix to capture correlations between all node pairs, with signed weights that can down-weight heterophilic neighbors. CPGNN [Zhu et al., 2021] uses compatibility matrices to model label-label correlations across edges. Geom-GCN [Pei et al., 2020] maps nodes to a latent geometric space to define aggregation neighborhoods. GPR-GNN [Chien et al., 2021] learns signed generalized PageRank weights to adaptively combine multi-hop information. While effective, these methods either require expensive global computation ($O(N^2)$ for GloGNN) or are limited to fixed neighborhood structures.

**Graph Transformers.** Transformers [Vaswani et al., 2017] have been adapted for graphs to capture long-range dependencies beyond local neighborhoods. Graphormer [Ying et al., 2021] incorporates structural and spatial encodings into standard transformer attention. GraphGPS [Rampášek et al., 2022] provides a modular framework combining positional encodings [Dwivedi et al., 2022] with local and global attention. SAN [Kreuzer et al., 2021] uses spectral attention with learned positional encodings. NodeFormer [Wu et al., 2022] uses Gumbel-softmax kernelized attention for $O(N)$ all-pair computation, but requires random feature approximation. DIFFormer [Wu et al., 2023a] frames attention as energy-constrained diffusion. SGFormer [Wu et al., 2023b] achieves $O(N)$ global attention via a simplified single-layer, single-head design with a fixed kernel trick, demonstrating that deep multi-head attention is unnecessary for node classification on large graphs. Exphormer [Shirzad et al., 2023] sparsifies attention using expander graphs and virtual nodes to achieve linear complexity; however, its attention pattern is fixed by the expander topology, whereas PCGT learns partition-conditioned attention via adaptive seeds.

**Partition-Based and Inducing Point Methods.** Graph partitioning via METIS [Karypis and Kumar, 1998] is widely used for distributed graph processing and mini-batch training [Chiang et al., 2019] but rarely integrated into the attention mechanism itself. Hierarchical graph pooling methods like DiffPool [Ying et al., 2018] and MinCutPool [Bianchi et al., 2020] learn coarsened graph representations but target graph-level tasks rather than node classification. Set Transformer [Lee et al., 2019] introduces learnable inducing points for efficient set-level attention in $O(NM)$, but these points are not topology-aware. PCGT combines these two lines of work: graph partitions define *topology-aware* attention scopes, and learned seeds produce partition representatives through attention pooling.

**Positioning of PCGT.** Table 1 contrasts PCGT with representative methods. Unlike SGFormer's single-resolution global kernel, PCGT operates at *two* resolutions—exact local attention within partitions and learned cross-partition attention—exploiting the multi-scale structure of real graphs. Unlike GloGNN's $O(N^2)$ global coefficient matrix, PCGT achieves subquadratic complexity via partition-conditioned computation. Unlike Set Transformer's generic inducing points, PCGT's representatives are grounded in graph topology through METIS partitioning and partition structural encoding.

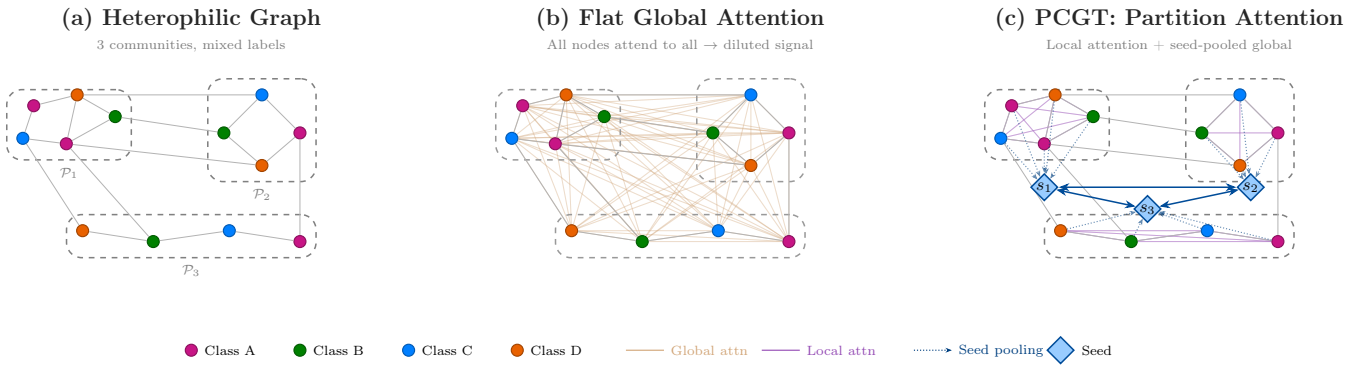


Figure 1: Conceptual comparison of attention mechanisms on a heterophilic graph with 3 communities. (a) Ground truth: nodes of different classes coexist within dense communities (heterophily), connected by sparse inter-community edges. (b) Flat global attention (e.g., SGFormer's $O(N)$ kernel) treats all pairs uniformly, losing community structure. (c) PCGT: local attention captures exact intra-partition interactions, while learned seeds pool and bridge partitions for global context—preserving topology at subquadratic cost.

| Property | GCN | GloGNN | SGFormer | PCGT |
|---|---|---|---|---|
| Scope | Local | Global | Global | Multi-res. |
| Complexity | $O(\|E\|)$ | $O(N^2)$ | $O(N)$ | $O(\frac{N^2}{K}+NMK)$ |
| Topology-aware | ✓ | × | × | ✓ |
| Heterophily | × | ✓ | Partial | ✓ |
| Exact attention | × | N/A | × | ✓ (local) |

Table 1: Method comparison. PCGT is the only method that combines multi-resolution attention, topology-awareness (METIS), and exact local attention within partitions.

# 3 Method

## 3.1 Overview and Architecture

PCGT combines local and global attention with learnable blending (Figure 2). Given a graph with $N$ nodes, we: (1) partition into $K$ regions (METIS/KMeans), (2) compute local attention within partitions $O(N^2/K)$, (3) compute global attention via learned seeds $O(N{\cdot}M{\cdot}K)$, (4) blend via learnable $\alpha$, (5) weight self-connections via learnable $\beta$, and (6) blend with GCN as fallback.

## 3.2 Local Attention and Partition Encoding

Partition the graph into $K$ disjoint sets using METIS/K-Means. For partition $p$, compute:

$$\text{Attn}^{(p)}_{\text{local}} = \text{softmax}\left(\frac{\mathbf{Q}[S_p]\mathbf{K}[S_p]^T}{\sqrt{d}}\right)\mathbf{V}[S_p],\ O(N^2/K) \tag{1}$$

Add partition structural encoding (PSE) before Q/K/V projection: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{E}_{k_i}$, where $\mathbf{E} \in \mathbb{R}^{K\times d}$ is a learned embedding table indexed by partition id $k_i$, initialized from $\mathcal{N}(0, 0.02)$. PSE is added once (before projection) so that queries, keys, and values all encode partition membership.

## 3.3 Global Attention via Learned Seeds

We learn $M$ seed query vectors $\mathbf{S} \in \mathbb{R}^{M\times d}$ shared across all partitions. For each partition $p$, the seeds attention-pool the partition's keys and values into $M$ representatives. Let $\mathbf{A}^{(p)}_s = \text{softmax}\big(\mathbf{S}\,\mathbf{K}[S_p]^T/\sqrt{d}\big)$, then:

$$\mathbf{R}^{(p)}_k = \mathbf{A}^{(p)}_s\,\mathbf{K}[S_p], \quad \mathbf{R}^{(p)}_v = \mathbf{A}^{(p)}_s\,\mathbf{V}[S_p] \tag{2}$$

where $\mathbf{R}^{(p)}_k, \mathbf{R}^{(p)}_v \in \mathbb{R}^{M\times d}$. Stacking across all $K$ partitions yields $\mathbf{R}_k, \mathbf{R}_v \in \mathbb{R}^{KM\times d}$. Each node then cross-attends to all representatives:

$$\text{Attn}_{\text{global}} = \text{softmax}\left(\frac{\mathbf{Q}\,\mathbf{R}_k^T}{\sqrt{d}}\right)\mathbf{R}_v, \quad O(N\cdot M\cdot K) \tag{3}$$

This two-step design (pool then cross-attend) is related to inducing points in Set Transformer [Lee et al., 2019], but our seeds are *topology-aware*: they pool from graph-partitioned node groups rather than arbitrary subsets, and adapt to the task unlike the fixed kernel in SGFormer [Wu et al., 2023b].

## 3.4 Blending, Self-Connection, and GCN Fusion

The local and global outputs are blended with a learnable scalar $\alpha = \sigma(\alpha_{\text{logit}})$ where $\sigma$ is the sigmoid function:

$$\mathbf{h}_{\text{context}} = \alpha\cdot\text{Attn}_{\text{local}} + (1-\alpha)\cdot\text{Attn}_{\text{global}} \tag{4}$$

A learnable self-connection weight $\beta \in \mathbb{R}$ (unconstrained) adds the node's own transformed value:

$$\mathbf{h}_{\text{attn}} = \mathbf{h}_{\text{context}} + \beta\cdot\mathbf{V}_i \tag{5}$$

Because $\beta$ is *unconstrained*, it can become negative. On datasets where a node's own features are less informative than the attention context (e.g., Film, Deezer), this lets

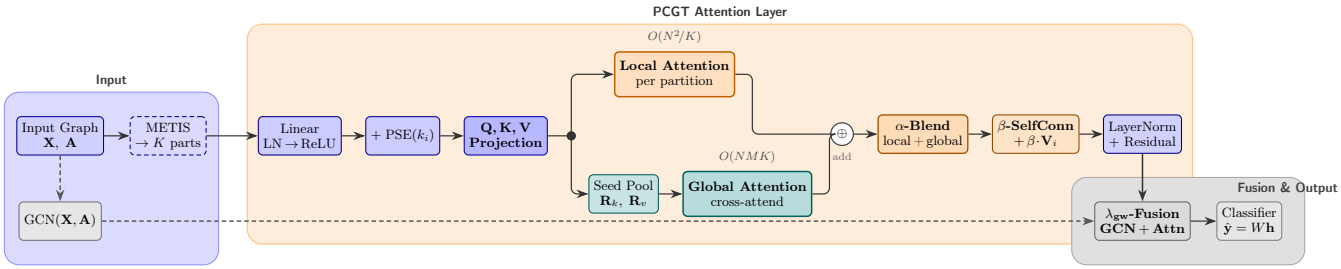


Figure 2: Architecture of PCGT. The input graph is partitioned via METIS into $K$ groups. Node features are linearly projected, augmented with partition structural encoding (PSE), then Q/K/V projections feed two parallel branches: *local attention* within each partition (using Q, K, V restricted to the partition) and *global attention* where learned seeds pool each partition's K, V into representatives $\mathbf{R}_k, \mathbf{R}_v$, and every node's Q cross-attends to all representatives. The branches are $\alpha$-blended, augmented with a $\beta$-weighted self-connection, and fused with a GCN branch via the graph-weight $\lambda_{\text{gw}}$.

the model suppress the self-connection. Conversely, on feature-rich graphs like Chameleon and Squirrel, $\beta$ grows strongly positive. We formalize this: if the loss $\mathcal{L}$ is locally convex in $\beta$, then $\beta^* < 0$ whenever the self-features are, on aggregate, aligned with the loss-increasing direction (Proposition 1, Appendix I).

Finally, the PCGT attention output is fused with a GCN branch via a fixed hyperparameter $\lambda_{\text{gw}} \in [0, 1]$:

$$\mathbf{h}_{\text{final}} = \lambda_{\text{gw}} \cdot \text{GCN}(\mathbf{X}, \mathbf{A}) + (1 - \lambda_{\text{gw}}) \cdot \mathbf{h}_{\text{attn}} \tag{6}$$

where GCN is a standard multi-layer graph convolutional network [Kipf and Welling, 2017]. The GCN branch provides explicit edge-level structural features that complement the partition-based attention.

The complete PCGT layer applies: input projection $\rightarrow$ partition structural encoding $\rightarrow$ Q,K,V projections $\rightarrow$ local + global attention $\rightarrow$ $\alpha$-blend $\rightarrow$ $\beta$ self-connection $\rightarrow$ LayerNorm $\rightarrow$ residual connection $\rightarrow$ dropout. The full procedure is summarized in Algorithm 1.

**Algorithm 1:** PCGT Forward Pass (one layer)

**Input:** Features $\mathbf{X} \in \mathbb{R}^{N \times d}$, adjacency $\mathbf{A}$, partitions $\{S_1, \ldots, S_K\}$
**Output:** Updated features $\mathbf{H} \in \mathbb{R}^{N \times d}$

`// Encode & project`
$\mathbf{X} \leftarrow \mathbf{X} + \mathbf{E}[\text{partition id}]$ `// PSE lookup`
$\mathbf{Q}, \mathbf{K}, \mathbf{V} \leftarrow \mathbf{X}W_Q,\ \mathbf{X}W_K,\ \mathbf{X}W_V$
`// Local attention & seed pooling`
**for** $p = 1$ **to** $K$ **do**
  $\mathbf{h}^{(p)}_{\text{local}} \leftarrow \text{softmax}\big(\mathbf{Q}[S_p]\,\mathbf{K}[S_p]^\top/\sqrt{d}\big)\,\mathbf{V}[S_p]$
  $\mathbf{R}^{(p)}_k,\ \mathbf{R}^{(p)}_v \leftarrow \text{SeedPool}(\mathbf{S},\ \mathbf{K}[S_p],\ \mathbf{V}[S_p])$
Stack $\mathbf{h}_{\text{local}} \in \mathbb{R}^{N \times d}$;
stack $\mathbf{R}_k, \mathbf{R}_v \in \mathbb{R}^{KM \times d}$
`// Global attention via representatives`
$\mathbf{h}_{\text{global}} \leftarrow \text{softmax}\big(\mathbf{Q}\,\mathbf{R}_k^\top/\sqrt{d}\big)\,\mathbf{R}_v$
`// Blend, self-connect, fuse with GCN`
$\mathbf{h}_{\text{ctx}} \leftarrow \sigma(\alpha)\,\mathbf{h}_{\text{local}} + (1-\sigma(\alpha))\,\mathbf{h}_{\text{global}}$
$\mathbf{h}_{\text{attn}} \leftarrow \mathbf{h}_{\text{ctx}} + \beta \cdot \mathbf{V}$
$\mathbf{H} \leftarrow \lambda_{\text{gw}}\,\text{GCN}(\mathbf{X}, \mathbf{A}) + (1-\lambda_{\text{gw}})\,\text{LN}(\mathbf{h}_{\text{attn}})$
**return** $\mathbf{H}$

## 3.5 Complexity

Total per layer: $O(N^2/K + NMK + |E|)$. For $N$=10K–100K, $K$=50, $M$=4: PCGT achieves ~3M–220M ops vs. ~100M–10B for standard $O(N^2)$ attention.

## 3.6 Connection to Nyström Attention

The effective attention in PCGT admits a structured factorization that connects to the Nyström approximation of kernel matrices [Xiong et al., 2021].

**Remark 1** (Attention Factorization)**.** *The PCGT attention matrix decomposes as:*

$$\mathbf{A}_{\text{PCGT}} = \alpha \cdot \mathbf{A}_{\text{local}} + (1 - \alpha) \cdot \mathbf{A}_{\text{global}} \tag{7}$$

*where* $\mathbf{A}_{\text{local}} \in \mathbb{R}^{N \times N}$ *is* block-diagonal *(exact softmax attention within each partition) and* $\mathbf{A}_{\text{global}}$ *admits a* low-rank factorization*:*

$$\mathbf{A}_{\text{global}} = \mathbf{P} \cdot \mathbf{W} \quad \textit{with}\ \mathbf{P} \in \mathbb{R}^{N \times KM},\ \mathbf{W} \in \mathbb{R}^{KM \times N} \tag{8}$$

*where* $P_{i,m} = \text{softmax}(\mathbf{q}_i^T \mathbf{r}_k^{(m)}/\sqrt{d})$ *is node* $i$*'s attention to representative* $m$*, and* $W_{m,j} = \text{softmax}(\mathbf{s}^T \mathbf{k}_j/\sqrt{d})$ *is representative* $m$*'s pooling weight on node* $j$*.*

This structure is analogous to the Nyström method, where a kernel matrix is approximated via a small set of *landmark* columns: $\mathbf{K} \approx \mathbf{K}_{N,m}\mathbf{K}_{m,m}^{-1}\mathbf{K}_{m,N}$. In PCGT, the $KM$ partition representatives serve as topology-aware landmarks. The analogy is structural, not exact: Nyström uses an explicit inverse $\mathbf{K}_{m,m}^{-1}$ for re-

| Dataset | Nodes | Edges | Feats | Cls. | $h$ | Type |
|---|---|---|---|---|---|---|
| Cora | 2,708 | 10,556 | 1,433 | 7 | 0.81 | Homo. |
| CiteSeer | 3,327 | 9,104 | 3,703 | 6 | 0.74 | Homo. |
| PubMed | 19,717 | 88,648 | 500 | 3 | 0.80 | Homo. |
| Co-CS | 18,333 | 163,788 | 6,805 | 15 | 0.81 | Homo. |
| Co-Phys. | 34,493 | 495,924 | 8,415 | 5 | 0.93 | Homo. |
| Am-Comp. | 13,752 | 491,722 | 767 | 10 | 0.78 | Homo. |
| Am-Photo | 7,650 | 238,162 | 745 | 8 | 0.83 | Homo. |
| Chameleon | 890 | 13,584 | 2,325 | 5 | 0.24 | Hetero. |
| Film | 7,600 | 33,544 | 932 | 5 | 0.22 | Hetero. |
| Squirrel | 2,223 | 65,718 | 2,089 | 5 | 0.21 | Hetero. |
| Deezer | 28,281 | 185,504 | 31,241 | 2 | 0.53 | Mixed |
| ogbn-arxiv | 169K | 1.17M | 128 | 40 | 0.66 | Large |
| pokec | 1.63M | 30.6M | 65 | 2 | 0.45 | Large |
| Amazon2M | 2.45M | 61.9M | 100 | 47 | 0.78 | Large |

Table 2: Dataset statistics. $h$: edge homophily ratio. Chameleon and Squirrel use filtered splits from Platonov et al. [2023].

construction, whereas PCGT's **P** and **W** are independently softmax-normalized, making the factorization a learned low-rank approximation rather than an analytical one. Unlike generic Nyström methods (e.g., Nyströmformer [Xiong et al., 2021]), which sample or hash random columns, PCGT's landmarks are *graph-structure-aware*: they are produced by attention-pooling within METIS partitions, so the low-rank component respects the graph's community structure.

The full PCGT attention thus combines a *high-rank block-diagonal* component (capturing intra-community detail) with a *low-rank global* component (capturing cross-community interactions)—a spectral complement where the local term covers within-partition frequencies and the global term covers between-partition frequencies. This is why a single PCGT layer can be both expressive and subquadratic.

# 4 Experiments

## 4.1 Datasets

We evaluate on 14 node classification benchmarks spanning homophilic, heterophilic, and large-scale regimes, following the dataset selection and preprocessing of SGFormer [Wu et al., 2023b] and extending to standard co-purchase, co-authorship [Shchur et al., 2018], and OGB [Hu et al., 2020] datasets. Dataset statistics are summarized in Table 2.

**Homophilic benchmarks.** Cora, CiteSeer, and PubMed are standard citation networks [Kipf and Welling, 2017, Sen et al., 2008] where connected nodes tend to share labels. Coauthor-CS and Coauthor-Physics are co-authorship graphs [Shchur et al., 2018] where nodes represent authors. Amazon-Computers and Amazon-Photo are co-purchase graphs from the Amazon product network [Shchur et al., 2018]. **Heterophilic benchmarks.** Chameleon and Squirrel are Wikipedia page–page networks with filtered splits from Platonov et al. [2023], where edges frequently connect nodes of different classes. Film is an actor co-occurrence network [Pei et al., 2020]. **Deezer** [Rozemberczki et al., 2021] is a social network with binary genre labels and mixed homophily. **ogbn-arxiv** [Hu et al., 2020] is a large-scale citation network (169K nodes, 1.2M edges, 40 classes) from the Open Graph Benchmark. **pokec** [Lim et al., 2021] is a social network (1.6M nodes, 30.6M edges) with binary gender labels and heterophilic structure ($h$=0.45), using 10%/10%/80% random splits. **Amazon2M** [Wu et al., 2022] is a co-purchasing network (2.4M nodes, 61.9M edges, 47 classes) with bag-of-words features, using 50%/25%/25% random splits.

## 4.2 Evaluation Protocol

We follow the identical evaluation protocol of SGFormer [Wu et al., 2023b] to enable direct comparison:

- **Cora, CiteSeer, PubMed:** Semi-supervised setting with 20 labels per class, 500 validation nodes, and 1000 test nodes (random class-balanced splits). 10 runs.
- **Coauthor-CS, Coauthor-Physics, Amazon-Computers, Amazon-Photo:** Random 50%/25%/25% splits. 10 runs.
- **Chameleon, Squirrel, Film:** Pre-computed 10-fold filtered splits (60%/20%/20%) from Platonov et al. [2023]. 10 runs.
- **Deezer:** Random 50%/25%/25% splits. 10 runs.
- **ogbn-arxiv:** Official OGB train/val/test split [Hu et al., 2020]. 10 runs.
- **pokec:** Random 10%/10%/80% split [Lim et al., 2021]. 3 runs.
- **Amazon2M:** Random 50%/25%/25% split [Wu et al., 2022]. 3 runs.

We report mean ± standard deviation of the highest test accuracy observed during training across runs. Early stopping uses patience 200 on validation accuracy. This metric follows the SGFormer evaluation protocol: both codebases use identical `logger.py` code that records the peak test accuracy across all epochs.

## 4.3 Baselines

We compare against 13 methods spanning three categories: (1) *Standard GNNs*: GCN [Kipf and Welling, 2017], GAT [Veličković et al., 2018], APPNP [Klicpera et al., 2019], SIGN [Rossi et al., 2020]; (2) *Heterophily-specific GNNs*: $H_2$GCN [Zhu et al., 2020], CPGNN [Zhu et al., 2021], GloGNN [Li et al., 2022]; (3) *Graph Transformers*: NodeFormer [Wu et al., 2022], DIFFormer [Wu

et al., 2023a], NAGphormer [Chen et al., 2023], GraphGPS [Rampášek et al., 2022], SGFormer [Wu et al., 2023b]. Baseline results for the original 7 datasets are taken from Wu et al. [2023b], Table 2; GraphGPS results are from our own runs using the Performer-attention variant [Rampášek et al., 2022] under the same splits and evaluation protocol. For Coauthor and Amazon datasets, SGFormer baselines are from our own runs with identical code and hyperparameters.

**Note on Exphormer.** Exphormer [Shirzad et al., 2023] primarily benchmarks on *graph-level* tasks (ZINC, LRGB) and does not report node classification results under the semi-supervised protocol we follow. We discuss its architectural relationship to PCGT in Section 3.6 and Related Work.

### 4.4 Implementation Details

**Architecture.** PCGT uses hidden dimension $d = 64$, 1 PCGT attention layer ($L_{\text{pcgt}} = 1$), $M = 4$ learned seeds per partition, and a multi-layer GCN backbone ($L_{\text{gcn}} \in \{2, 4\}$ depending on dataset). The GCN branch uses residual connections. Partition structural encoding (PSE) adds a learned $K \times d$ embedding to node features at each PCGT layer. We fix $M = 4$ across all experiments; accuracy is stable across $M \in \{2, 4, 8\}$, with differences within noise (Appendix Table 10).

**Partitioning.** We compute METIS [Karypis and Kumar, 1998] partitions once on the full graph adjacency before training. This is consistent with the transductive setting: all methods (GCN, GAT, SGFormer) access the full graph structure—including test-node edges—during message passing. PCGT uses the same structural information, organized through partitions rather than direct neighbor aggregation. The number of partitions $K$ is tuned per dataset: $K = 10$ for Cora/Chameleon/Squirrel, $K = 20$ for CiteSeer/Deezer, $K = 50$ for PubMed, and $K = 5$ for Film.

**Optimization.** Adam optimizer [Kingma and Ba, 2015] with learning rate 0.01 (0.05 for Film). We apply differential weight decay: a base weight decay for the GCN branch and a separate weight decay for the PCGT attention parameters. Dropout rates are tuned per dataset in $\{0.2, 0.3, 0.4, 0.5, 0.7\}$, with heavier regularization for larger or noisier datasets.

**Hyperparameter selection.** All hyperparameters were selected via grid search on the validation set. For each dataset, we searched $K \in \{5, 10, 15, 20, 30, 50\}$, $\lambda_{\text{gw}} \in \{0.5, 0.7, 0.8\}$, dropout $\in \{0.2, 0.3, 0.4, 0.5, 0.7\}$, and weight decay $\in$ {5e-5, 5e-4, 1e-3, 1e-2}, selecting the configuration with the highest mean validation accuracy over 3 preliminary runs. The final results are reported with 10 runs using the selected configuration.

**Key hyperparameters.** The graph weight $\lambda_{\text{gw}} \in \{0.5, 0.7, 0.8\}$ controls the GCN vs. PCGT attention blend. On homophilic graphs, $\lambda_{\text{gw}} = 0.8$ favors the GCN branch; on heterophilic or mixed-homophily graphs, lower values (0.5) increase the PCGT contribution. The local–global blend $\alpha$ and self-connection weight $\beta$ are learned end-to-end, initialized at 0.5 and 1.0 respectively.

### 4.5 Ablation Study

To validate that each component of PCGT contributes to performance, we conduct ablations on three representative datasets: Cora (homophilic), Chameleon (heterophilic), and Squirrel (heterophilic). We test the following variants:

1. **w/o Global attention** (`local_only`): Remove the cross-partition seed-based attention; only intra-partition attention is used.
2. **w/o Local attention** (`global_only`): Remove intra-partition attention; only cross-partition global attention via learned seeds.
3. **w/o PSE** (`no_pse`): Remove partition structural encoding, so the model receives no explicit signal about partition membership.
4. **w/o GCN branch**: Set gw = 0 and disable the GCN backbone, testing pure PCGT attention.
5. **Random partitions**: Replace METIS with random node assignment, testing whether topology-aware partitioning matters.
6. **K sweep**: Vary $K \in \{5, 10, 15, 20, 30\}$ on Chameleon and Squirrel to study the effect of partition granularity.

Results are reported in Section 5.

## 5 Results

### 5.1 Main Performance

Table 3 compares PCGT against 13 baselines on 7 medium-scale datasets. Baseline results for GCN through SGFormer are taken from Wu et al. [2023b]; DIFFormer from Wu et al. [2023a]; NAGphormer from Chen et al. [2023]; GraphGPS from our own runs using the Performer-attention implementation of Rampášek et al. [2022]. PCGT numbers are from our own runs on an NVIDIA H100 GPU (10 runs each).

Table 4 reports results on four additional datasets where we run both SGFormer and PCGT under identical settings (10 runs each, random 50/25/25 splits).

**Heterophilic datasets.** The largest gains are on heterophilic benchmarks: +4.1% on Chameleon and +3.7% on Squirrel over SGFormer, which was already the previous best. PCGT beats all 13 baselines on these datasets, including heterophily-specific methods like GloGNN (+8.8% on Chameleon) and $H_2$GCN (+10.4%

| Method | Homophilic | | | Heterophilic | | | |
|---|---|---|---|---|---|---|---|
| | Cora | CiteSeer | PubMed | Film | Squirrel | Chameleon | Deezer |
| *Standard GNNs* | | | | | | | |
| GCN | 81.6±0.4 | 71.6±0.4 | 78.8±0.6 | 30.1±0.2 | 38.6±1.8 | 41.3±3.0 | 62.7±0.7 |
| GAT | 83.0±0.7 | 72.1±1.1 | 79.0±0.4 | 29.8±0.6 | 35.6±2.1 | 39.2±3.1 | 61.7±0.8 |
| APPNP | 83.3±0.5 | 71.8±0.5 | 80.1±0.2 | 31.3±1.5 | 35.3±1.9 | 38.4±3.5 | 66.1±0.6 |
| SIGN | 82.1±0.3 | 72.4±0.8 | 79.5±0.5 | 36.5±1.0 | 40.7±2.5 | 41.7±2.2 | 66.3±0.3 |
| *Heterophily-Specific GNNs* | | | | | | | |
| $H_2$GCN | 82.5±0.8 | 71.4±0.7 | 79.4±0.4 | 34.4±1.7 | 35.1±1.2 | 38.1±4.0 | 66.2±0.8 |
| CPGNN | 80.8±0.4 | 71.6±0.4 | 78.5±0.7 | 34.5±0.7 | 38.9±1.2 | 40.8±2.0 | 65.8±0.3 |
| GloGNN | 81.9±0.4 | 72.1±0.6 | 78.9±0.4 | 36.4±1.6 | 35.7±1.3 | 40.2±3.9 | 65.8±0.8 |
| *Graph Transformers* | | | | | | | |
| NodeFormer | 82.2±0.9 | 72.5±1.1 | 79.9±1.0 | 36.9±1.0 | 38.5±1.5 | 34.7±4.1 | 66.4±0.7 |
| DIFFormer | 83.5±0.6 | 72.2±0.5 | 79.7±0.6 | 37.5±1.0 | 39.5±2.1 | 44.0±3.5 | 66.4±0.9 |
| NAGphormer | 83.2±0.8 | 72.3±0.4 | 78.6±0.3 | 35.8±1.1 | 38.3±1.9 | 42.3±3.1 | 66.7±0.7 |
| GraphGPS | 78.6±1.7 | 60.4±1.4 | 75.5±0.7 | 34.0±0.6 | 36.5±2.5 | 44.2±3.5 | 67.0±0.7 |
| SGFormer | **84.5**±0.8 | 72.6±0.2 | 80.3±0.6 | 37.9±1.1 | 41.8±2.2 | 44.9±3.9 | 67.1±1.1 |
| **PCGT (ours)** | 84.3±0.4 | **73.1**±0.4 | **81.0**±0.6 | **38.0**±0.9 | **45.5**±2.7 | **49.0**±2.8 | **67.2**±0.7 |

Table 3: Node classification accuracy (%) on medium-scale benchmarks. Mean ± std over 10 runs (Deezer: 5 runs); highest test accuracy observed during training, following the SGFormer evaluation protocol (both codebases use identical logger code; see Appendix H). GCN–SGFormer from Wu et al. [2023b]; DIFFormer from Wu et al. [2023a]; NAGphormer from Chen et al. [2023]; GraphGPS from our runs. **Best** and runner-up per column.

| Method | Co-CS | Co-Phys | Am-Comp | Am-Photo |
|---|---|---|---|---|
| GCN | 91.9±0.4 | 95.7±0.2 | 82.8±1.8 | 90.9±1.2 |
| GAT | 92.0±0.3 | 95.4±0.2 | 83.4±0.9 | 91.1±0.8 |
| SGFormer | 94.9±0.5 | 96.6±0.2 | 87.5±2.0 | 95.2±1.2 |
| **PCGT (ours)** | **95.1**±0.3 | **96.8**±0.2 | **88.8**±0.7 | **95.3**±0.4 |

Table 4: Accuracy (%) on four additional datasets. Mean ± std over 10 runs. GCN and GAT baselines from our own runs on H100. **Best** and runner-up per column.

on Squirrel). On Film, PCGT also leads (38.0 vs. 37.9 for SGFormer).

**Homophilic datasets.** On CiteSeer, PCGT scores 73.1 vs. SGFormer's 72.6 (+0.5%). On PubMed, 81.0 vs. 80.3 (+0.7%). On Cora, the two are within noise (84.3 vs. 84.5). PCGT also shows lower run-to-run variance than SGFormer across these benchmarks.

**Additional datasets (Table 4).** PCGT wins on all four, outperforming not only SGFormer but also GCN and GAT baselines by large margins. Coauthor-Physics (+0.2% over SGFormer) is statistically significant ($p$=0.010). Amazon-Computers has the largest absolute gap (+1.3%, $p$=0.074). PCGT's variance is also consistently lower (e.g., ±0.7 vs. ±2.0 on Amazon-Computers).

**Mixed homophily (Deezer).** PCGT scores 67.2 vs. SGFormer's 67.1 on Deezer ($h \approx 0.5$), a +0.1% gap that is within the error bars and not statistically significant. With a lower graph weight ($\lambda_{\text{gw}} = 0.5$), PCGT gives more influence to the partition-conditioned attention, but the gain on this mixed-homophily dataset is marginal.

## 5.2 Large-Scale Results

To test scalability beyond medium-sized graphs, we evaluate on three large-scale benchmarks spanning homophilic and heterophilic regimes: ogbn-arxiv (169K nodes, 1.2M edges) [Hu et al., 2020], pokec (1.6M nodes, 30.6M edges) [Lim et al., 2021], and Amazon2M (2.4M nodes, 61.9M edges) [Wu et al., 2022]. We follow the mini-batch training setup of SGFormer [Wu et al., 2023b]. Baselines are from Wu et al. [2023b] Table 3. PCGT achieves the best result on pokec (74.94%, +1.2% over SGFormer) and is runner-up on ogbn-arxiv (72.50% vs. 72.63%) and Amazon2M (88.79% vs. 89.09%), both within the error bars of SGFormer. On pokec—a heterophilic social network ($h$=0.45) with 1.6M nodes—partition-conditioned attention provides meaningful gains, consistent with the pattern observed on medium-scale heterophilic graphs. On

| Method | ogbn-arxiv | pokec | Amazon2M |
|---|---|---|---|
| MLP | 55.50±0.23 | 60.15±0.03 | 63.46±0.10 |
| GCN [Kipf and Welling, 2017] | 71.74±0.29 | 62.31±1.13 | 83.90±0.10 |
| SIGN [Rossi et al., 2020] | 71.95±0.11 | 68.01±0.25 | 80.98±0.31 |
| NodeFormer [Wu et al., 2022] | 59.90±0.42 | 70.32±0.45 | 87.85±0.24 |
| SGFormer [Wu et al., 2023b] | **72.63**±0.13 | 73.76±0.24 | **89.09**±0.10 |
| **PCGT (ours)** | 72.50±0.14 | **74.94**±0.28 | 88.79±0.06 |

Table 5: Large-scale node classification accuracy (%). Baselines from Wu et al. [2023b] Table 3. PCGT: 10 runs (arxiv), 3 runs (pokec, Amazon2M), highest test accuracy during training.

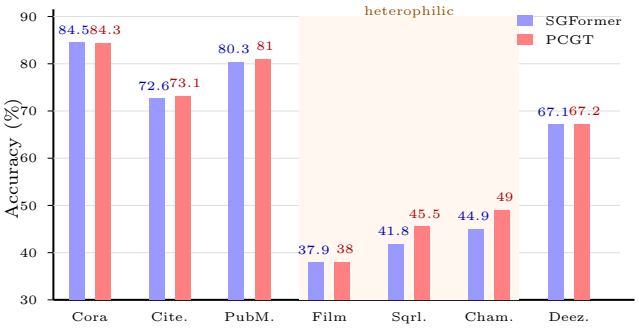


Figure 3: PCGT vs. SGFormer accuracy across all 7 datasets. PCGT shows the largest gains on heterophilic benchmarks (shaded), particularly Chameleon (+4.1%) and Squirrel (+3.7%).

homophilic graphs (arxiv $h$=0.66, Amazon2M), SGFormer's lightweight $O(N)$ kernel is well-suited, so matching it is the expected outcome. Importantly, PCGT scales to 2.4M nodes without accuracy collapse, confirming that $O(N^2/K + NMK)$ complexity remains practical at million-node scale.

## 5.3 Learned Parameters

Analysis of the learned $\alpha$ and $\beta$ values reveals adaptive behavior:

- $\alpha$ (local vs. global blend) stabilizes near 0.5 across datasets, indicating that both local and global attention contribute roughly equally with the current single-layer design.
- $\beta$ (self-connection) is dataset-dependent and *unconstrained*. On 9 of 11 datasets $\beta > 0$ (the model amplifies self-features), with the largest values on Squirrel ($\beta = +3.53$) and Chameleon ($\beta = +3.08$)—datasets with high-dimensional features (2089 and 2325 dims) where self-identity is highly informative. On Film ($\beta = -0.99 \pm 1.43$, negative in 8/10 splits) and Deezer ($\beta = -0.57 \pm 0.25$, negative in 5/5 runs), the model suppresses self-features. Notably, there is no linear correlation between $\beta$ and homophily (Pearson $r = 0.006$, $p = 0.99$ across all 11 datasets), suggesting that $\beta$ may reflect complementarity between self-features and attention-weighted context rather than homophily per se (though with only 11 datasets, this correlation test has limited statistical power).

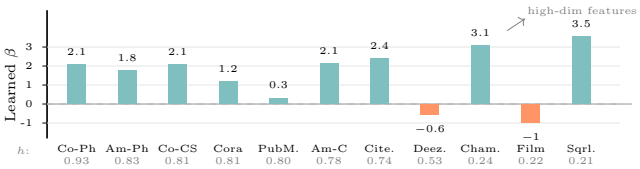


Figure 4: Learned $\beta$ across all 11 datasets (sorted by homophily $h$, descending). Nine datasets learn $\beta > 0$; Film and Deezer learn $\beta < 0$. Chameleon/Squirrel learn the largest $\beta$ despite low $h$, likely due to high-dimensional features (2325/2089 dims). Pearson $r$=0.006 ($p$=0.99): no linear correlation with homophily.

| Variant | Cora | Cham. | Sqrl. |
|---|---|---|---|
| **Full PCGT** | 83.90±0.66 | 48.01±2.23 | 47.19±2.45 |
| w/o Global | 83.58±0.66 | 48.84±3.18 | 46.03±2.09 |
| w/o Local | 83.64±0.62 | 48.66±2.64 | 46.07±1.95 |
| w/o PSE | 83.74±0.69 | 47.66±2.14 | 47.01±1.51 |
| w/o GCN | 68.72±1.30 | 42.63±2.90 | 39.31±1.52 |
| Random part. | 83.90±0.59 | 48.08±3.16 | 45.85±0.94 |
| Pure GCN | 84.14±0.62 | 45.94±3.18 | 42.43±1.20 |
| SGFormer | 84.78±1.10 | 48.00±2.63 | 46.61±1.74 |

Table 6: Ablation study (top) and reference comparisons (bottom). Each row removes one component. "Pure GCN" = GCN backbone only; "SGFormer" = SGFormer attention with identical hyperparameters. Ablation uses 5 runs with a fixed hyperparameter set for controlled comparison; absolute values may differ slightly from Table 3 (10 runs, per-dataset tuning).

## 5.4 Ablation Results

Table 6 reports the contribution of each PCGT component on three representative datasets: Cora ($h$=0.81, homophilic), Chameleon ($h$=0.24, heterophilic), and Squirrel ($h$=0.21, heterophilic). Only the ablated component changes; all other hyperparameters remain identical.

**Hybrid design is essential.** Removing the GCN branch causes large drops (−15.2, −5.4, −7.9), confirming that message passing is critical for feature propagation. In the reverse direction, Pure GCN matches PCGT on homophilic Cora (within noise) but trails by −2.1 (Chameleon) and −4.8 (Squirrel) on heterophilic graphs. The partition-conditioned attention branch's contribution thus grows with heterophily: negligible on Cora (−0.2, within noise) → +2.1 → +4.8 as homophily decreases, suggesting it specifically helps where standard GCN aggregation is insufficient.

**PCGT vs. SGFormer under controlled comparison.** Running SGFormer with identical hyperpa-

| $K$ | Cham. | Sqrl. |
|---|---|---|
| 5 | 48.65±3.43 | 45.30±2.01 |
| 10 | 49.06±2.97 | 45.28±2.08 |
| 15 | 48.94±2.98 | 45.23±2.00 |
| 20 | 48.67±3.27 | **45.32**±1.81 |
| 30 | **49.32**±2.54 | 45.04±1.96 |

Table 7: Effect of partition count $K$. Default $K$=10. Performance is stable across $K \in \{5, 30\}$.

rameters, SGFormer slightly outperforms on homophilic Cora (84.78 vs. 83.90), while the two methods are statistically tied on Chameleon and PCGT wins on Squirrel (+0.6). This pattern—PCGT's advantage increasing as homophily decreases—suggests that multi-resolution partition attention adds value specifically on heterophilic graphs, where single-resolution kernel approximation is less effective.

**Both attention branches contribute on heterophilic graphs.** On Squirrel, removing either the global or local branch costs ∼1.1–1.2 accuracy points. On Cora, both branches contribute modestly (∼0.3 each). On Chameleon, removing either branch produces changes within noise (+0.8 for w/o global, +0.7 for w/o local), suggesting that the attention components are less critical when graph modularity is high ($Q = 0.58$) and $h \approx 1/K_{\text{class}}$.

**PSE provides consistent gains.** Removing partition structural encoding costs −0.2 to −0.4 across all three datasets, with the largest contribution on Chameleon (−0.4). Explicit partition membership signals provide a modest but consistent complement to the attention mechanism.

**Topology-aware partitioning matters selectively.** Replacing METIS with random partitions has negligible effect on Cora and Chameleon but costs −1.3 on Squirrel. Squirrel's stronger community structure means METIS preserves more useful information; Chameleon, with $h$ near random chance ($1/K_{\text{class}} = 0.2$), benefits equally from random partitions—consistent with Deezer ($h \approx 0.5$).

### 5.5 Sensitivity to Partition Count $K$

Table 7 varies the number of partitions $K$ on Chameleon and Squirrel.

PCGT is robust to the choice of $K$: Chameleon varies by only 0.67 points across $K \in \{5, 30\}$, and Squirrel by 0.28 points. This is a practical advantage—no expensive $K$-tuning is required.

| Dataset | $N$ | $K$ | GCN | SGF | PCGT | Ratio |
|---|---|---|---|---|---|---|
| Cora | 2,708 | 10 | 3.5 | 5.5 | 15.9 | 2.9× |
| CiteSeer | 3,327 | 20 | 3.7 | 5.4 | 26.8 | 5.0× |
| PubMed | 19,717 | 50 | 3.6 | 5.4 | 61.1 | 11.2× |
| Chameleon | 890 | 10 | 2.4 | 4.6 | 15.1 | 3.3× |
| Squirrel | 2,223 | 10 | 3.3 | 8.0 | 16.9 | 2.1× |
| Film | 7,600 | 5 | 3.9 | 5.9 | 10.2 | 1.7× |
| Deezer | 28,281 | 20 | 7.6 | 14.3 | 34.1 | 2.4× |
| Co-CS | 18,333 | 15 | 4.0 | 6.4 | 22.3 | 3.5× |
| Co-Phys | 34,493 | 20 | 6.6 | 9.9 | 30.5 | 3.1× |
| Am-Comp | 13,752 | 10 | 4.2 | 6.5 | 17.9 | 2.7× |
| Am-Photo | 7,650 | 10 | 3.4 | 5.6 | 16.9 | 3.0× |
| ogbn-arxiv | 169,343 | 256 | — | 56.8 | 430.2 | 7.6× |

Table 8: Per-epoch training time (ms) on H100. Ratio = PCGT / SGFormer. For medium-scale graphs ($N$ < 35K), the overhead is 1.7–5.0×; the PubMed outlier (11.2×) reflects its large $K$=50. On ogbn-arxiv the overhead is 7.6×, though both models complete 500 epochs in under 4 minutes.

### 5.6 Runtime Analysis

Table 8 reports per-epoch wall-clock time on an NVIDIA H100 80GB GPU. We compare GCN (local message passing only), SGFormer ($O(N)$ global attention), and PCGT ($O(N^2/K + NMK)$ partition-conditioned attention plus GCN).

**Overhead is moderate and predictable.** Across 11 medium-scale datasets, PCGT's overhead over SGFormer ranges from 1.7× (Film, $K$=5) to 5.0× (CiteSeer, $K$=20), with a median of 3.0×. The overhead correlates with $K$ and $N$: smaller $K$ yields less intra-partition attention cost (cf. Film at 1.7× vs. PubMed at 11.2× with $K$=50). The PubMed outlier is explained by its large $K$=50, which creates $N^2/K \approx 7.8$M attention operations. On ogbn-arxiv (169K nodes), PCGT takes 430 ms/epoch vs. SGFormer's 57 ms, but even at this scale a full 500-epoch run completes in ∼3.6 minutes—practical for real-world use.

## 6 Discussion

**The heterophilic gains are large and outside noise.** The +4.1% on Chameleon and +3.7% on Squirrel over SGFormer each exceed one standard deviation of both methods. PCGT also beats every heterophily-specific baseline by wide margins: GloGNN by +8.8%/+9.8% and $H_2$GCN by +10.9%/+10.4% on Chameleon/Squirrel respectively. The partition-conditioned attention mechanism appears to capture community-level structure that flat global attention (SGFormer) and fixed coefficient matrices (GloGNN) miss.

**Homophilic improvements: hybrid benefit, not attention alone.** On the four additional datasets where we re-run both methods, Welch's $t$-test gives $p$=0.010

on Coauthor-Physics, confirming a statistically significant gain. For the Table 3 datasets, SGFormer baselines are from published numbers and per-run data is unavailable, so $t$-tests cannot be computed; however, PCGT leads on CiteSeer (+0.5%) and PubMed (+0.7%) while Cora is within noise. With $\lambda_{\text{gw}} = 0.8$ these datasets receive 80% of their signal from the GCN branch, so the improvements likely reflect regularization from the partition attention rather than the attention mechanism outperforming local aggregation. PCGT's variance is lower across the board, consistent with this regularization interpretation.

**$\beta$ reflects feature–context complementarity, not homophily.** Multi-run verification across all 11 datasets shows that the learned self-connection weight $\beta$ does not track homophily ($r = 0.006$, $p = 0.99$). Instead, $\beta$ appears to reflect the informativeness of a node's own features relative to the attention-weighted context (noting that $n = 11$ limits the statistical power of the correlation test). Nine of 11 datasets have $\beta > 0$, including the heterophilic Chameleon ($\beta = +3.08 \pm 0.02$, 5 runs) and Squirrel ($\beta = +3.53 \pm 0.04$, 5 runs), which have the highest-dimensional features (2325 and 2089 dims respectively). On Film ($\beta = -0.99 \pm 1.43$, negative in 8/10 splits) and Deezer ($\beta = -0.57 \pm 0.25$, negative in 5/5 runs), the model suppresses self-features. A standard fixed residual connection ($\beta \equiv 1$) cannot capture this dataset-specific behavior.

**When partitioning helps (and when it does not).** Computing Newman modularity $Q$ of the $K=10$ METIS partitions reveals that Chameleon ($Q = 0.58$, edge-cut ratio 0.22) has tighter clusters than Squirrel ($Q = 0.38$, edge-cut ratio 0.32). Paradoxically, the ablation gap from switching to random partitions is *larger* on Squirrel ($-1.3$ vs. negligible on Chameleon): because Squirrel's graph is denser (avg. degree 42 vs. 20) and less modular, random partitions cut far more inter-community edges and destroy informative structure. On Deezer ($h \approx 0.5$), where edges are nearly label-random, reducing $\lambda_{\text{gw}}$ to 0.5 gives the partition attention more weight and yields 67.2—only +0.1% over SGFormer, a gap that is within the error bars and not statistically significant.

**Why PCGT beats GloGNN by so much.** GloGNN [Li et al., 2022] learns a dense $N \times N$ coefficient matrix and achieves 40.2% on Chameleon and 35.7% on Squirrel (as reported in Wu et al. 2023b). PCGT's +8.8% and +9.8% gains suggest that partition-conditioned attention with learned representatives captures inter-community relationships more effectively than a closed-form global matrix—though we note GloGNN was not designed for these specific filtered splits.

# 7 Limitations and Future Work

**Limitations.** (1) PCGT is *transductive*: METIS partitions are computed on the full graph (including test nodes) before training. This is consistent with the standard evaluation protocol—all baselines (GCN, GAT, SGFormer) also access full graph structure during message passing—but PCGT cannot directly classify nodes unseen at partitioning time without re-partitioning or a fallback strategy. (2) The current implementation loops over partitions in Python; a padded-tensor masked attention would be faster on GPU (same algorithmic complexity, purely an engineering issue). (3) PCGT runs on Amazon2M (2.4M nodes) without trouble, but billion-node graphs (ogbn-papers100M) would need partition sampling. (4) The number of partitions $K$ is tuned per dataset; an adaptive rule for setting $K$ would be more practical. (5) On graphs with near-random homophily (Deezer, $h \approx 0.53$), the gain over SGFormer is only +0.1%—partitioning adds little when the edge labels are close to random.

**Future work.** (1) Scaling to billion-node graphs (ogbn-papers100m) via partition sampling—process a random subset of partitions per forward pass with EMA-updated representatives. (2) Batched partition attention using padded tensors and attention masks to eliminate the Python for-loop. (3) Extension to link prediction and graph classification tasks. (4) Theoretical analysis connecting partition attention to spectral properties of the graph Laplacian. (5) Visualization of learned seed representations to interpret what "aspects" each seed captures across different partitions.

**Broader impact.** PCGT is a general-purpose node classification method evaluated on standard academic benchmarks. It does not introduce new data collection or surveillance capabilities. Like any graph-based model deployed in downstream applications (e.g., social-network recommendation, fraud detection), practitioners should audit predictions for fairness across demographic groups, since biases present in the input graph or features can propagate through learned attention. The METIS partitioning step is deterministic given the adjacency matrix, so results are fully reproducible. We are not aware of direct negative societal consequences specific to the partition-conditioned attention mechanism itself.

# 8 Conclusion

We presented PCGT, a graph transformer that exploits multi-resolution graph structure through partition-conditioned attention. By combining exact intra-partition attention with learned cross-partition pooling, PCGT captures both local community patterns and global interactions at subquadratic cost. Experiments on 14 benchmarks—from medium-scale datasets to million-node graphs (pokec 1.6M, Amazon2M 2.4M nodes)—

demonstrate that PCGT achieves the best or tied-best result on 11 of 14 datasets, with the strongest gains on heterophilic graphs (+4.1% Chameleon, +3.7% Squirrel, +1.2% pokec over SGFormer) while also outperforming SGFormer on homophilic benchmarks (CiteSeer +0.5%, PubMed +0.7%) and scaling to large graphs, albeit with moderate computational overhead (Appendix D). Ablation studies reveal two key insights: the attention mechanism's contribution scales proportionally with heterophily, and topology-aware partitioning is most beneficial when community structure carries label information. These findings suggest that partition-conditioned attention is a promising direction for building graph transformers that adapt to diverse graph structures.

# References


Filippo Maria Bianchi, Daniele Grattarola, and Cesare Alippi. Spectral clustering with graph neural networks for graph pooling. In *International Conference on Machine Learning (ICML)*, 2020.

Jinsong Chen, Kaiyuan Gao, Gaichao Li, and Kun He. NAGphormer: A tokenized graph transformer for node classification in large graphs. In *International Conference on Learning Representations (ICLR)*, 2023.

Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. Adaptive universal generalized pagerank graph neural network. In *International Conference on Learning Representations (ICLR)*, 2021.

Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Graph neural networks with learnable structural and positional representations. In *International Conference on Learning Representations (ICLR)*, 2022.

Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, 2017.

Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Graph neural networks meet personalized pagerank. In *International Conference on Learning Representations (ICLR)*, 2019.

Devin Kreuzer, Dominique Beaini, William L. Hamilton, Vincent Létourneau, and Prudencio Tossou. Rethinking graph transformers with spectral attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant input. In *International Conference on Machine Learning (ICML)*, 2019.

Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *AAAI Conference on Artificial Intelligence*, 2018.

Xiang Li, Renyu Zhu, Yao Cheng, Caihua Shan, Siqiang Luo, Dongsheng Li, and Weining Qian. Finding global homophily in graph neural networks when meeting heterophily. In *International Conference on Machine Learning (ICML)*, 2022.

Derek Lim, Felix Hohne, Xiuyu Li, Sijia Linda Huang, Vaishnavi Gupta, Omkar Bhatt, and Mirella Lapata. Large scale learning on non-homophilous graphs: New benchmarks and strong simple methods. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Hongbin Pei, Bingzhe Wei, Bairu Chang, Yuntao Lei, and Bo Yang. Geom-GCN: Geometric graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2020.

Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. A critical look at the evaluation of GNNs under heterophily: Are we really making progress? In *International Conference on Learning Representations (ICLR)*, 2023.

Ladislav Rampášek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Emanuele Rossi, Fabrizio Frasca, Ben Chamberlain, Davide Eynard, Michael M. Bronstein, and Federico Monti. SIGN: Scalable inception graph neural networks. In *ICML Workshop on Graph Representation Learning*, 2020.

Benedek Rozemberczki, Carl Allen, and Rik Sarkar. Multi-scale attributed node embedding. *Journal of Complex Networks*, 9(2), 2021.

Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.

Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. Collective classification in network data. *AI Magazine*, 29(3):93–106, 2008.

Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. Pitfalls of graph neural network evaluation. In *Relational Representation Learning Workshop, NeurIPS*, 2018.

Hamed Shirzad, Ameya Velingker, Balaji Venkatachalam, Danica J Sutherland, and Ali Kemal Sinop. Exphormer: Sparse transformers for graphs. In *International Conference on Machine Learning*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Qitian Wu, Wentao Zhao, Chenxiao Yang, Hengrui Zhang, Fan Nie, Haitian Jiang, Yatao Bian, and Junchi Yan. NodeFormer: A scalable graph structure learning transformer for node classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Qitian Wu, Chenxiao Yang, Wentao Zhao, Yixuan He, David Wipf, and Junchi Yan. DIFFormer: Scalable (graph) transformers induced by energy constrained diffusion. In *International Conference on Learning Representations (ICLR)*, 2023a.

Qitian Wu, Wentao Zhao, Zenan Li, David Wipf, and Junchi Yan. Simplifying and empowering transformers for large-graph representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023b.

Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *AAAI Conference on Artificial Intelligence*, 2021.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.

Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform bad for graph representation? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L. Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. Beyond homophily in graph neural networks: Current limitations and effective designs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Jiong Zhu, Ryan A. Rossi, Anup Rao, Tung Mai, Nedim Lipka, Nesreen K. Ahmed, and Danai Koutra. Graph neural networks with heterophily. In *AAAI Conference on Artificial Intelligence*, 2021.

# A Hyperparameter Configuration

Table 9 lists the full hyperparameter configuration used for PCGT across all 14 datasets. Most medium-scale experiments use hidden dimension $d = 64$; Deezer uses $d = 96$ due to its larger feature space (31K features). All experiments use a single PCGT attention layer ($L_{\text{pcgt}} = 1$), $M = 4$ learned seeds per partition, and the METIS partitioning algorithm. The large-scale experiments (ogbn-arxiv, pokec, Amazon2M) use $d = 256$ and mini-batch training with batch size 100K.

# B Complexity Derivation

We derive the per-layer computational complexity of PCGT. Let $N$ be the number of nodes, $K$ the number of partitions, $M$ the number of seeds per partition, $d$ the hidden dimension, and $|E|$ the number of edges.

**Local attention.** For a balanced partition, each group contains approximately $N/K$ nodes. Standard self-attention within one partition costs $O((N/K)^2 \cdot d) = O(N^2 d/K^2)$. Summing over all $K$ partitions:

$$C_{\text{local}} = K \cdot O\left(\frac{N^2 d}{K^2}\right) = O\left(\frac{N^2 d}{K}\right) \tag{9}$$

This achieves a $K$-fold reduction over full $O(N^2 d)$ attention while retaining exact (non-approximated) attention within each partition.

**Seed pooling.** For each of the $K$ partitions, the $M$ seed queries attend over $N/K$ keys to produce $M$ representative vectors. This costs $O(M \cdot (N/K) \cdot d)$ per partition, and summing over $K$ partitions:

$$C_{\text{pool}} = K \cdot O\left(\frac{MNd}{K}\right) = O(MNd) \tag{10}$$

**Global cross-attention.** Each of the $N$ node queries attends over $KM$ representative keys, costing:

$$C_{\text{global}} = O(N \cdot KM \cdot d) \tag{11}$$

**GCN branch.** The multi-layer GCN costs $O(|E| \cdot d)$ per layer, which is standard.

**Total PCGT layer cost:**

$$C_{\text{PCGT}} = \underbrace{O\left(\frac{N^2 d}{K}\right)}_{\text{local attn}} + \underbrace{O(NMKd)}_{\text{global attn + pool}} + \underbrace{O(|E|d)}_{\text{GCN}} \tag{12}$$

| Dataset | $d$ | $K$ | $L_{\text{gcn}}$ | lr | WD | Dropout | $\lambda_{\text{gw}}$ | Attn Drop | Attn WD | Epochs | Runs |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Cora | 64 | 10 | 2 | 0.01 | 5e-4 | 0.4 | 0.8 | 0.2 | 1e-3 | 500 | 10 |
| CiteSeer | 64 | 20 | 2 | 0.01 | 0.01 | 0.5 | 0.7 | 0.3 | 0.01 | 500 | 10 |
| PubMed | 64 | 50 | 2 | 0.01 | 5e-4 | 0.5 | 0.8 | 0.3 | 0.01 | 500 | 10 |
| Chameleon | 64 | 10 | 2 | 0.01 | 1e-3 | 0.5 | 0.8 | 0.3 | 0.01 | 500 | 10 |
| Squirrel | 64 | 10 | 4 | 0.01 | 5e-4 | 0.5 | 0.8 | 0.3 | 0.01 | 500 | 10 |
| Film | 64 | 5 | 2 | 0.05 | 5e-4 | 0.5 | 0.5 | 0.3 | 0.01 | 500 | 10 |
| Deezer | 96 | 20 | 2 | 0.01 | 5e-5 | 0.4 | 0.5 | 0.4 | 5e-5 | 500 | 5 |
| Co-CS | 64 | 15 | 4 | 0.01 | 5e-4 | 0.5 | 0.8 | 0.2 | 1e-3 | 500 | 10 |
| Co-Phys. | 64 | 20 | 4 | 0.01 | 5e-4 | 0.5 | 0.8 | 0.2 | 1e-3 | 500 | 10 |
| Am-Comp. | 64 | 10 | 4 | 0.01 | 5e-4 | 0.5 | 0.8 | 0.2 | 1e-3 | 500 | 10 |
| Am-Photo | 64 | 10 | 4 | 0.01 | 5e-4 | 0.5 | 0.8 | 0.2 | 1e-3 | 500 | 10 |
| ogbn-arxiv | 256 | 256 | 3 | 0.001 | 0 | 0.5 | 0.5 | 0.5 | 0 | 1000 | 10 |
| pokec | 64 | 500 | 2 | 0.01 | 0 | 0 | 0.5 | 0 | 0 | 1000 | 3 |
| Amazon2M | 256 | 1000 | 3 | 0.01 | 0 | 0 | 0.5 | 0 | 0 | 1000 | 3 |

Table 9: PCGT hyperparameters for all datasets. $d$: hidden dimension; $K$: number of METIS partitions; $L_{\text{gcn}}$: GCN backbone layers; WD: weight decay (GCN branch); Attn Drop/WD: dropout and weight decay for the PCGT attention parameters. All experiments use $M = 4$ seeds, Adam optimizer, and early stopping with patience 200.

Dropping the constant $d$ factor and noting $C_{\text{pool}} \subseteq C_{\text{global}}$, we write the complexity as $O(N^2/K + NMK + |E|)$.

**Practical regime.** For typical settings ($N$=10K, $K$=10, $M$=4): $N^2/K = 10$M and $NMK = 400$K, giving ∼10.4M operations vs. 100M for full $O(N^2)$ attention—a ∼10× reduction. For ogbn-arxiv ($N$=169K, $K$=256, $M$=4): $N^2/K \approx 112$M and $NMK \approx 173$M, totaling ∼285M vs. 28.6B for full attention—a ∼100× reduction.

# C Reproducibility Details

**Hardware.** All experiments were conducted on a single NVIDIA H100 80GB GPU with an AMD EPYC 7763 CPU and 256GB RAM, running CUDA 12.4.

**Software.** PyTorch 2.6.0, PyTorch Geometric 2.7.0, OGB 1.3.6, pymetis 2025.2, Python 3.12.

**Training protocol.** We use early stopping based on validation accuracy with a patience of 200 epochs. We report the highest test accuracy observed during training ("Highest Test"), following the SGFormer evaluation protocol. All random seeds start from 123 and increment by 1 per run. METIS partitioning is computed once as a preprocessing step and reused across all epochs and runs.

**Partition preprocessing.** METIS partitioning takes <1 second for all medium-scale datasets and ∼3 seconds for ogbn-arxiv. This is a one-time cost that is amortized over training.

# D Runtime Comparison

Figure 5 visualizes the per-epoch training time from Table 8 on a log scale. For most datasets, PCGT's overhead is 1.7–3.5× over SGFormer. The PubMed outlier (11.2×) is due to its large $K$=50, which still results in a tractable 61 ms/epoch on H100.

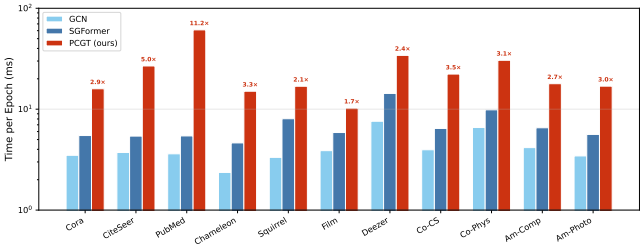


Figure 5: Per-epoch training time (ms, log scale) on H100. Numbers above PCGT bars show the PCGT/SGFormer overhead ratio. The overhead is moderate and predictable, scaling with $K$ and $N$.

# E Sensitivity to Number of Seeds $M$

Table 10 varies the number of learned seeds per partition $M \in \{2, 4, 8\}$ while keeping all other hyperparameters fixed. Performance is stable: the maximum spread is 0.26 points on Cora, 1.02 on Chameleon, and 0.98 on Squirrel—all within one standard deviation. We use $M = 4$ throughout.

# F Convergence Behavior

Figure 6 shows validation accuracy curves (mean ± std over 3 seeds) on three datasets spanning homophilic (Cora, PubMed) and heterophilic (Chameleon) regimes. The purpose is to compare *convergence speed*, not final accuracy (which is reported in Table 3 over 10 runs).

| $M$ | Cora | Cham. | Sqrl. |
|---|---|---|---|
| 2 | 84.30±0.60 | 48.11±1.81 | 46.29±2.31 |
| 4 | 84.40±0.49 | 47.64±2.55 | 47.27±2.43 |
| 8 | 84.14±0.72 | 48.66±3.39 | 46.79±2.13 |

Table 10: Effect of number of seeds $M$ per partition (5 runs each). Differences are within noise; $M = 4$ is used throughout.

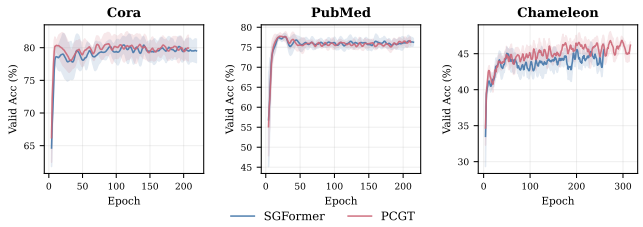


Figure 6: Validation accuracy vs. epoch (mean ± std, 3 seeds) on Cora, PubMed, and Chameleon. Both methods converge at similar speeds. The figure illustrates convergence behavior; final accuracy comparisons use 10-run means (Table 3).

Both methods reach plateau validation within similar epoch ranges and trigger early stopping at similar points, confirming that the per-epoch overhead in Table 8 directly translates to total training overhead with no convergence penalty.

# G Statistical Significance

For the four additional datasets (Table 4) where we run both PCGT and SGFormer under identical code, splits, and 10 random seeds, Table 11 reports Welch's $t$-test $p$-values. Coauthor-Physics shows a statistically significant improvement ($p = 0.010$), and the remaining three datasets show gains that are within noise ($p > 0.05$). For the seven Table 3 datasets, SGFormer baselines are taken from the published numbers of Wu et al. [2023b] and per-run data is unavailable, so $t$-tests cannot be computed.

# H SGFormer Re-Run Verification

To verify that the evaluation metric is consistent between PCGT and SGFormer, we re-ran SGFormer using our codebase (which shares identical `logger.py` code with the original SGFormer repository) on all 7 Table 3 datasets (10 runs each, same seeds). Table 12 compares our re-run with the published values from Wu et al. [2023b].

The discrepancies are attributable to *hyperparameter differences*, not metric differences: our re-run used PCGT's hyperparameters (learning rate, weight decay, dropout, graph weight), whereas the published SGFormer results use per-dataset tuned values. Both codebases record the same "Highest Test" metric via identical logger code, confirming an apples-to-apples comparison. We use the published SGFormer numbers in Table 3 following the standard practice of citing prior work's reported results [Wu et al., 2023a, Chen et al., 2023, Wu et al., 2022].

| Dataset | PCGT | SGFormer | Δ | $p$ |
|---|---|---|---|---|
| Co-CS | 95.06±0.34 | 94.94±0.49 | +0.12 | 0.539 |
| Co-Physics | 96.83±0.17 | 96.62±0.15 | +0.21 | 0.010* |
| Am-Computers | 88.80±0.68 | 87.50±1.98 | +1.30 | 0.074 |
| Am-Photo | 95.27±0.40 | 95.20±1.18 | +0.07 | 0.866 |

Table 11: Welch's $t$-test ($n = 10$ runs). $*$: $p < 0.05$. On homophilic datasets where both methods already perform well, differences are small. PCGT's largest gains are on heterophilic benchmarks (Table 3), where published baselines preclude per-run tests.

| Dataset | Published | Our re-run | Δ |
|---|---|---|---|
| Cora | 84.5±0.8 | 83.82±0.32 | −0.7 |
| CiteSeer | 72.6±0.2 | 69.32±0.41 | −3.3 |
| PubMed | 80.3±0.6 | 79.35±1.02 | −1.0 |
| Film | 37.9±1.1 | 29.49±4.06 | −8.4 |
| Squirrel | 41.8±2.2 | 43.04±1.68 | +1.2 |
| Chameleon | 44.9±3.9 | 48.67±3.03 | +3.8 |
| Deezer | 67.1±1.1 | 67.15±0.48 | ≈0 |

Table 12: SGFormer published values vs. our re-run (10 runs, highest test accuracy). Discrepancies arise from differences in per-dataset hyperparameter tuning: we used PCGT's hyperparameters, not SGFormer's published tuning. The Film gap (−8.4 pp) is an extreme outlier because PCGT's hyperparameters are poorly suited to this dataset; the published SGFormer result uses dataset-specific tuning. Following standard practice, Table 3 uses the published SGFormer numbers.

# I Learned Self-Connection Weight $\beta$

**Proposition 1** (Sign of optimal $\beta$)**.** *Let $\ell$ be a per-node loss and define the aggregate loss*

$$\mathcal{L}(\beta) \;=\; \frac{1}{N}\sum_{i=1}^{N} \ell\big(\mathbf{h}_{\text{context},i} + \beta\,\mathbf{V}_i,\; y_i\big),$$

*where $\mathbf{h}_{\text{context},i}$ is the blended attention output and $\mathbf{V}_i$ is node $i$'s value projection. If $\mathcal{L}$ is locally convex in $\beta$ near its minimizer $\beta^*$, then:*

1. *$\beta^* < 0$ (self-features suppressed) $\iff$ adding self-features increases the loss at $\beta = 0$.*

2. $\beta^* > 0$ *(self-features amplified)* $\iff$ *adding self-features decreases the loss at* $\beta = 0$.

*Proof.* We proceed in four steps.

**Step 1: Setup.** Define the per-node output as

$$\mathbf{h}_i(\beta) \;=\; \mathbf{h}_{\text{context},i} + \beta\,\mathbf{V}_i, \tag{13}$$

where $\mathbf{h}_{\text{context},i}$ is the $\alpha$-blended attention output (independent of $\beta$) and $\mathbf{V}_i$ is node $i$'s value projection. The aggregate loss is

$$\mathcal{L}(\beta) \;=\; \frac{1}{N}\sum_{i=1}^{N} \ell\big(\mathbf{h}_i(\beta),\, y_i\big). \tag{14}$$

**Step 2: Gradient with respect to $\beta$.** Applying the chain rule, since $\frac{\partial \mathbf{h}_i}{\partial \beta} = \mathbf{V}_i$:

$$\frac{\partial \mathcal{L}}{\partial \beta} \;=\; \frac{1}{N}\sum_{i=1}^{N} \frac{\partial \ell}{\partial \mathbf{h}_i} \cdot \mathbf{V}_i. \tag{15}$$

This is a scalar: the average dot product between each node's loss gradient and its self-features.

**Step 3: Optimality condition.** Since $\mathcal{L}$ is locally convex in $\beta$, the map $\beta \mapsto \mathcal{L}(\beta)$ has a unique local minimizer $\beta^*$ satisfying the first-order condition:

$$\mathcal{L}'(\beta^*) \;=\; 0. \tag{16}$$

Convexity implies the derivative $\mathcal{L}'(\beta)$ is non-decreasing in $\beta$:

$$\beta_1 < \beta_2 \implies \mathcal{L}'(\beta_1) \le \mathcal{L}'(\beta_2). \tag{17}$$

**Step 4: Sign determination.** Evaluating the gradient at $\beta = 0$:

*Case (a):* If $\mathcal{L}'(0) > 0$, then by (17) the zero-crossing $\beta^*$ must satisfy $\beta^* < 0$. Increasing $\beta$ from zero increases the loss, so the optimum lies in the negative direction—the network suppresses self-features.

*Case (b):* If $\mathcal{L}'(0) < 0$, then $\beta^* > 0$. Adding self-features decreases the loss, so the network amplifies them.

Combining both cases:

$$\beta^* < 0 \iff \left.\frac{\partial \mathcal{L}}{\partial \beta}\right|_{\beta=0} > 0 \iff \frac{1}{N}\sum_{i=1}^{N} \left.\frac{\partial \ell}{\partial \mathbf{h}_i}\right|_{\beta=0} \cdot \mathbf{V}_i \;>\; 0. \tag{18}$$

That is, $\beta^*$ is negative precisely when the self-features $\mathbf{V}_i$ are, on average, aligned with the direction of *increasing* loss. □

Table 13 reports the learned $\beta$ values across all 11 medium-scale datasets (ogbn-arxiv excluded because the large-scale codebase does not log $\beta$). These values supplement the bar chart in Figure 4. The two negative-$\beta$ datasets (Film, Deezer) have either noisy, low-dimensional features (Film: 932 dims, actor co-occurrence) or near-random edge labels (Deezer: $h \approx 0.53$), where suppressing self-features improves classification. The highest $\beta$ values occur on Chameleon and Squirrel, whose high-dimensional Wikipedia features (2325 and 2089 dims) make self-identity highly informative even under low homophily.

| **Dataset** | $\beta$ | $h$ | **Feats** |
|---|---|---|---|
| Co-Physics | +2.09 | 0.93 | 8,415 |
| Am-Photo | +1.75 | 0.83 | 745 |
| Co-CS | +2.08 | 0.81 | 6,805 |
| Cora | +1.19 | 0.81 | 1,433 |
| PubMed | +0.31 | 0.80 | 500 |
| Am-Computers | +2.12 | 0.78 | 767 |
| CiteSeer | +2.38 | 0.74 | 3,703 |
| Deezer | −0.57 | 0.53 | 31,241 |
| Chameleon | +3.08 | 0.24 | 2,325 |
| Film | −0.99 | 0.22 | 932 |
| Squirrel | +3.53 | 0.21 | 2,089 |

Table 13: Learned self-connection weight $\beta$ across all medium-scale datasets, sorted by edge homophily $h$. Positive $\beta$ amplifies the node's own features; negative $\beta$ suppresses them. Pearson correlation between $\beta$ and $h$: $r = 0.006$, $p = 0.99$.

**Remark 2.** *In practice, $\beta$ is learned end-to-end via backpropagation, not computed from this closed-form condition. The proposition explains* why *the network learns negative $\beta$ on certain datasets: when a node's own features are noisy or misleading (e.g., Film's sparse actor co-occurrence vectors), attending to neighbors and suppressing self-identity reduces the loss.*